

# Canonical Case, Ledger & Evidence

Detailed architecture and implementation specification

United States deployment baseline

|          |                                                                                      |
|----------|--------------------------------------------------------------------------------------|
| CASE     | Transactional current business state, domain invariants and governed commands        |
| LEDGER   | Ordered immutable business history with sequence, hash chains and audit exports      |
| EVIDENCE | Content-addressed artifacts, provenance, lineage, retention and legal hold           |
| AZURE    | PostgreSQL Flexible Server, Blob Storage, Container Apps, Service Bus and Event Grid |

Planning baseline | Version 1.0 | 17 September 2026

Defines the durable source-of-truth boundary for foundation and KYC cases, decisions, lineage and evidence. Configuration values are proposed and require target-subscription validation.

# Contents

|                                                  |    |
|--------------------------------------------------|----|
| Purpose and outcome                              | 4  |
| 1. Mission, boundary, and invariants             | 4  |
| 2. Logical architecture                          | 5  |
| 3. Sources of truth                              | 6  |
| 4. Canonical Case aggregate                      | 6  |
| 5. Relational data model                         | 7  |
| 6. Case lifecycle                                | 7  |
| 7. BusinessCommand contract                      | 7  |
| 8. Case Command Gateway                          | 8  |
| 9. Transaction design                            | 8  |
| 10. Concurrency and merge semantics              | 9  |
| 11. Idempotency                                  | 9  |
| 12. CaseEvent envelope                           | 9  |
| 13. Ledger storage and tamper evidence           | 10 |
| 14. Event schemas and evolution                  | 10 |
| 15. Evidence object model                        | 11 |
| 16. Evidence intake lifecycle                    | 11 |
| 17. Blob artifact layout                         | 12 |
| 18. Immutable storage, retention, and legal hold | 12 |
| 19. Evidence, Claim, Fact, and Finding           | 12 |
| 20. Bitemporal facts                             | 13 |
| 21. Lineage graph                                | 13 |
| 22. Assumptions and contradictions               | 14 |
| 23. Decision chain                               | 14 |
| 24. Human review                                 | 14 |
| 25. Query and read models                        | 14 |
| 26. Search and retrieval                         | 15 |
| 27. APIs                                         | 15 |
| 28. Events and outbox                            | 16 |
| 29. Security and authorization                   | 16 |
| 30. Encryption and key management                | 16 |
| 31. Privacy, minimization, and erasure           | 16 |
| 32. Azure PostgreSQL configuration               | 17 |
| 33. Azure Blob configuration                     | 17 |
| 34. Service topology and sizing                  | 18 |
| 35. Observability and reconciliation             | 18 |
| 36. Backup, recovery, and regional continuity    | 19 |
| 37. KYC domain rules                             | 19 |
| 38. CI/CD and database change management         | 20 |

|                                                   |    |
|---------------------------------------------------|----|
| 39. Implementation plan                           | 20 |
| 40. Prioritized engineering backlog               | 21 |
| 41. Verification strategy                         | 22 |
| 42. Operational runbooks                          | 22 |
| 43. Architecture decisions and definition of done | 22 |

# Purpose and outcome

The Canonical Case Service, Case Ledger, and Evidence Store form the durable business and epistemic core of the Harness Factory. The Case Service answers what is currently believed and operationally effective. The Case Ledger answers how that state changed, who or what caused it, under which authority, and from which prior state. The Evidence Store preserves the source material and transformations that support claims, facts, findings, recommendations, gates, and final dispositions.

This United States baseline extends the foundation, KYC domain, Harness Control Plane, Execution Control Plane, Agent Runtime, and Capability Gateway specifications. It assumes Azure Database for PostgreSQL Flexible Server for transactional state and ledger metadata, Azure Blob Storage for content-addressed evidence artifacts, Azure Container Apps for services, Microsoft Entra managed identities, private networking, and Azure Service Bus/Event Grid for asynchronous propagation. Configuration values are proposed and require validation in target subscriptions.

## 1. Mission, boundary, and invariants

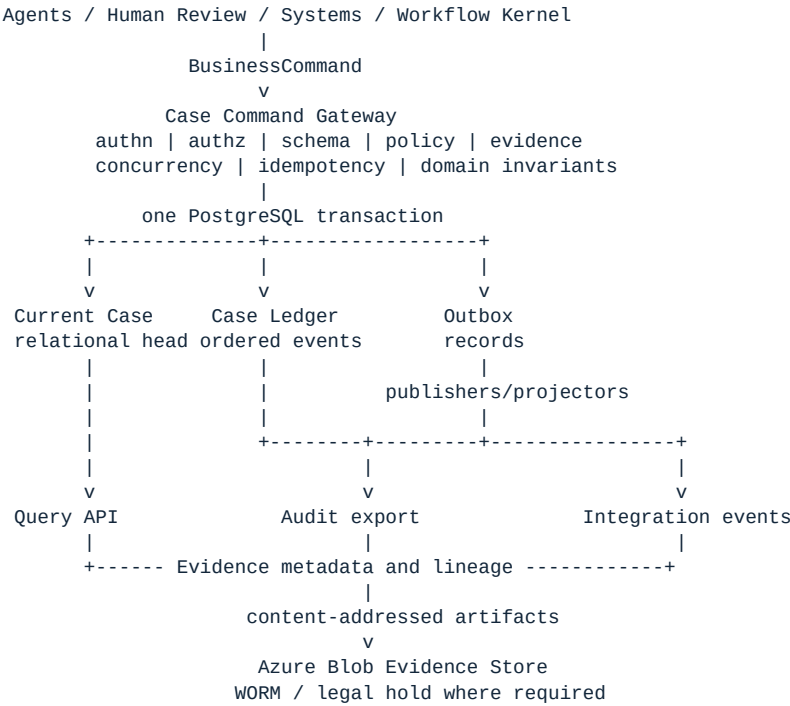
The platform needs one authoritative business head and one immutable history. Neither an agent conversation, model transcript, workflow-engine variable bag, vector index, search corpus, Foundry session, nor event stream is the canonical case.

The Canonical Case Service owns the current case aggregate, domain invariants, concurrency, accepted BusinessCommands, and effective dispositions. The Case Ledger owns the ordered record of every material accepted transition. The Evidence Store owns immutable content and provenance. The Execution Record owns computational history. Operational telemetry owns service behavior. Each record has a distinct purpose and retention model.

The following invariants are enforced:

- Agents, analysts, integrations, and workflow services change a case only through versioned BusinessCommands.
- Every accepted material mutation and its CaseEvent commit in the same PostgreSQL transaction.
- A case has one monotonically increasing sequence. No committed sequence is reused or reordered.
- Ledger events are append-only. Corrections create new events and state, never destructive history edits.
- Evidence content is immutable and addressed by a cryptographic digest. Metadata changes create new versions or annotations.
- A finding must cite evidence, facts, or accepted claims. A final disposition must cite a recommendation, deterministic gate result, policy snapshot, and actor authority.
- Evidence, claim, fact, finding, assumption, contradiction, recommendation, gate result, and disposition remain different object types.
- Case reads are tenant-bound and purpose-authorized. Cross-case or cross-tenant access is denied unless an explicit governed process allows it.
- Idempotency is based on semantic business operation, not delivery attempt.
- Current state can be reconstructed or reconciled from a trusted snapshot plus the ordered ledger.
- Search and analytics projections can be deleted and rebuilt without losing business truth.
- Retention, legal hold, residency, erasure restrictions, and access classification are stored with every evidence artifact and governed record.

## 2. Logical architecture



Derived only: Azure AI Search, analytics lake, graph projections, caches

| Component            | Responsibility                                                                                 | Authoritative record                       |
|----------------------|------------------------------------------------------------------------------------------------|--------------------------------------------|
| Case Command Gateway | Authenticate, authorize, validate, evaluate policy/evidence, check concurrency and idempotency | Command acceptance or rejection            |
| Case domain service  | Execute aggregate rules and create normalized mutations/events                                 | Current case state                         |
| Case query API       | Return purpose-limited views and sequence/ETag                                                 | None; reads current state and projections  |
| Ledger writer        | Allocate sequence, build event envelope, chain event digest, append event                      | Ordered immutable CaseEvent metadata       |
| Evidence intake      | Validate source, malware state, content digest, classification, metadata, and retention        | Evidence object registration               |
| Artifact writer      | Stream bytes, verify digest, encrypt, and apply immutability policy                            | Immutable Blob artifact                    |
| Lineage service      | Maintain typed Evidence-to-Disposition relationships                                           | Provenance edges and state                 |
| Outbox publisher     | Publish committed facts to Service Bus/Event Grid                                              | Delivery cursor; source remains PostgreSQL |
| Projection workers   | Build search, graph, notification, and analytics views                                         | Rebuildable derived state                  |
| Audit exporter       | Export signed ledger segments and manifests to immutable storage                               | Independent audit copy                     |

### 3. Sources of truth

The design intentionally uses complementary canonical records.

| Record                | Question answered                                               | Storage                                         |
|-----------------------|-----------------------------------------------------------------|-------------------------------------------------|
| Current Case          | What is effective now?                                          | PostgreSQL relational aggregate                 |
| Case Ledger           | How did effective state evolve and under what authority?        | PostgreSQL ordered events plus immutable export |
| Evidence graph        | What was observed, asserted, accepted, interpreted, and why?    | PostgreSQL metadata/edges plus Blob content     |
| Execution Record      | What computation produced a proposal?                           | Runtime metadata plus Blob artifacts            |
| Operational telemetry | How did services behave?                                        | Azure Monitor and Application Insights          |
| Search projection     | How can authorized users retrieve case information efficiently? | Azure AI Search or database projection          |

The ledger is not a dump of every internal log. It records accepted material business changes. Rejected commands, model attempts, transient retries, and infrastructure diagnostics belong in command audit, execution records, or telemetry unless regulation or investigation policy promotes them to a ledger event.

### 4. Canonical Case aggregate

The Case is an aggregate root rather than one giant JSON document. Core fields are relational and domain extensions use schema-governed JSONB only where controlled evolution is valuable.

```
{
  "caseId": "case_01...",
  "caseType": "kyc_onboarding",
  "schemaVersion": "kyc.case.v4",
  "tenantId": "tenant_01...",
  "jurisdiction": "US",
  "status": "INVESTIGATING",
  "caseSequence": 482,
  "policySnapshotDigest": "sha256:...",
  "harnessPlanDigest": "sha256:...",
  "subjects": [],
  "risk": {},
  "claims": [],
  "facts": [],
  "evidence": [],
  "findings": [],
  "assumptions": [],
  "contradictions": [],
  "workItems": [],
  "decisionRecommendations": [],
  "gateResults": [],
  "finalDispositions": [],
  "reviews": [],
  "executionRefs": [],
  "createdAt": "...",
  "updatedAt": "..."
}
```

The aggregate boundary includes data that must be consistent for command acceptance. Large content, execution payloads, and search documents remain outside the transaction and are referenced by verified digest. Cross-case relationships use explicit link objects with authorization and purpose rather than implicit foreign-key traversal exposed to callers.

## 5. Relational data model

```

case_core.cases
case_core.case_subjects
case_core.case_subject_identifiers
case_core.case_risk
case_core.case_claims
case_core.case_facts
case_core.case_findings
case_core.case_assumptions
case_core.case_contradictions
case_core.case_investigations
case_core.case_decision_recommendations
case_core.case_gate_results
case_core.case_final_dispositions
case_core.case_reviews
case_core.case_execution_refs
case_core.case_evidence_refs

case_ledger.case_events
case_ledger.command_receipts
case_ledger.event_chain_heads
case_ledger.ledger_segments
case_ledger.audit_exports

evidence.evidence_objects
evidence.artifacts
evidence.transformations
evidence.lineage_edges
evidence.retention_assignments
evidence.legal_holds

```

Every table includes tenant ID, case ID where applicable, schema version, created time, created actor, revision, classification, and provenance reference. Primary identifiers are globally unique and opaque. External identifiers are stored as typed aliases with source and tenant scope, never used as global primary keys.

Referential integrity is enforced for active transactional relationships. Historical payloads keep immutable identifiers and digests so the ledger remains interpretable after current rows are superseded. Database migrations are expand-and-contract and preserve event readers for every retained schema version.

## 6. Case lifecycle

The foundation lifecycle is CREATED, COLLECTING, VERIFYING, INVESTIGATING, DECISION\_READY, UNDER\_REVIEW, DISPOSITIONED, CLOSED, and ARCHIVED. Domain overlays may add substates but cannot bypass mandatory gates.

A lifecycle transition is a BusinessCommand evaluated against current sequence, subject revisions, required WorkItems, evidence freshness, open contradictions, policy snapshot, review requirements, and actor authority. A status field is never updated directly.

KYC reopening creates a new transition and reason. It may reactivate the same case or create a linked periodic-review case according to policy. A closed case is not silently mutated because a new vendor result arrived. New evidence is either appended under an allowed post-close event or routed to a new WorkItem/case.

## 7. BusinessCommand contract

BusinessCommands express intent and domain meaning. They do not expose physical table paths.

```

{
  "commandId": "cmd_01...",
  "commandType": "RecordScreeningFinding",
  "commandVersion": "kyc.command.record_screening_finding.v2",
  "tenantId": "tenant_01...",
  "caseId": "case_01...",
  "actor": {
    "type": "AGENT",
    "principalId": "screening-agent@2.3.0",

```

```

    "executionId": "exec_01..."
  },
  "authority": {
    "planDigest": "sha256:...",
    "permissionEnvelopeDigest": "sha256:...",
    "policySnapshotDigest": "sha256:..."
  },
  "payload": {
    "findingType": "pep_possible_match",
    "subjectRef": "subject_01...",
    "confidence": {"value": 0.81, "method": "model_assessed"},
    "evidenceRefs": ["ev_01...", "ev_02..."],
    "reason": "Two material identifiers matched the candidate record."
  },
  "preconditions": {
    "caseSequence": 482,
    "subjectRevision": 17
  },
  "idempotencyKey": "tenant:case:workitem:record-pep-finding:subject"
}

```

Command responses are ACCEPTED, REJECTED, CONFLICT, or IN\_PROGRESS. Rejections carry stable reason codes, rule identifiers, failed preconditions, current sequence, and safe remediation. Sensitive evidence or policy content is not copied into the error.

## 8. Case Command Gateway

The command path is deterministic and ordered:

1. Authenticate Entra principal and validate token audience and tenant.
2. Authorize command type, case, actor class, role, and compiled PermissionEnvelope.
3. Validate command and payload schemas.
4. Load the case aggregate at the current sequence inside a transaction.
5. Resolve idempotency and return the prior result when the semantic operation already completed.
6. Check sequence, entity revisions, WorkItem lease/fencing, and command preconditions.
7. Verify evidence existence, tenant/case scope, integrity, classification, freshness, provenance, and admissibility.
8. Evaluate domain invariants and the pinned policy snapshot.
9. Generate normalized CaseMutations and material CaseEvents.
10. Update current tables, append ledger events, advance sequence, create command receipt, and write outbox rows in one commit.
11. Return accepted sequence, created object IDs, event IDs, and aggregate ETag.

Agents never receive SQL access. Analyst applications and integrations use the same gateway so human actions remain under authorization, evidence, policy, concurrency, idempotency, and audit controls.

## 9. Transaction design

Use PostgreSQL transactions at READ COMMITTED with explicit aggregate locking or optimistic compare-and-swap, selected per command. A short transaction loads only the needed aggregate rows, validates preconditions, applies normalized mutations, inserts events, updates the chain head, writes outbox records, and commits.

```

begin;

select case_sequence
from case_core.cases
where tenant_id = $1 and case_id = $2
for update;

-- Verify expected sequence, command idempotency, entity revisions,
-- policy snapshot, evidence references, and domain conditions.

-- Apply normalized relational mutations.

```

```
-- Insert one or more case_events with consecutive sequence numbers.
-- Update event_chain_heads and cases.case_sequence.
-- Insert command_receipt and outbox messages.
```

```
commit;
```

Network or storage calls are prohibited inside the transaction. Evidence artifacts must already be staged and verified before a command references them. After commit, outbox consumers perform asynchronous projection, notification, and audit export.

## 10. Concurrency and merge semantics

A single case sequence provides total order, while entity revisions avoid unnecessary conflicts. Commands declare preconditions and one of the approved merge semantics.

| Merge semantic        | Example                     | Behavior on newer case sequence                                                   |
|-----------------------|-----------------------------|-----------------------------------------------------------------------------------|
| COMMUTATIVE_APPEND    | Append independent evidence | Accept if referenced subject exists and admissibility still holds                 |
| REBASE                | Add analyst note            | Reload and reapply if no protected field changed                                  |
| REVALIDATE_DEPENDENTS | Replace normalized identity | Accept only after marking dependent screenings/findings stale and creating rework |
| RECOMPUTE             | Risk score based on facts   | Reject result and schedule new computation                                        |
| EXACT_MATCH           | Final disposition           | Reject unless all sequence and gate preconditions still match                     |

The service never silently uses last-write-wins for regulated facts, contradictions, recommendations, or dispositions. A stale append can be accepted only when its semantic independence is proven. Conflicts return the current sequence and machine-readable rebase guidance.

## 11. Idempotency

Every command carries a semantic idempotency key scoped to tenant, case, command type, WorkItem or human task, and logical target. The command receipt stores request digest, outcome, accepted sequence range, created identifiers, and response digest.

A duplicate key with the same request digest returns the recorded response. A duplicate key with different content is a conflict. A caller retry after timeout cannot create repeated screenings, findings, reviews, escalations, notifications, or dispositions.

Command receipts are retained at least as long as the case and ledger events they protect. Delivery-layer message IDs and Service Bus duplicate detection are additional controls, never the source of business idempotency.

## 12. CaseEvent envelope

Every material event is immutable, versioned, and self-describing enough to audit without joining mutable current rows.

```
{
  "eventId": "evt_01...",
  "tenantId": "tenant_01...",
  "caseId": "case_01...",
  "caseSequence": 483,
  "eventType": "finding.created",
  "eventSchema": "kyc.finding.created.v2",
  "occurredAt": "2026-09-17T18:41:12.481Z",
  "recordedAt": "2026-09-17T18:41:12.519Z",
  "actor": {"type": "AGENT", "principalId": "screening-agent@2.3.0", "executionId": "exec_01..."},
  "commandRef": "cmd_01...",
  "authority": {
```

```

    "planDigest": "sha256:...",
    "policySnapshotDigest": "sha256:...",
    "permissionEnvelopeDigest": "sha256:..."
  },
  "payload": { "findingId": "finding_01...", "classification": "possible_pep_match" },
  "evidenceRefs": [ "ev_01...", "ev_02..." ],
  "previousEventDigest": "sha256:...",
  "eventDigest": "sha256:..."
}

```

occurredAt represents the domain-effective time supplied under policy; recordedAt is the database commit time. The event digest covers canonical envelope bytes excluding only the digest field itself. The previous digest creates a per-case chain. Periodic signed segment manifests anchor many case chains for independent tamper evidence.

## 13. Ledger storage and tamper evidence

PostgreSQL holds ledger metadata because current-state mutation and event append require one ACID transaction. Append permissions are exposed only through stored procedures or the Case Service role. Application identities have no update or delete rights on ledger tables. Database administrators remain privileged, so tamper evidence and independent immutable export are required.

```

create table case_ledger.case_events (
  tenant_id uuid not null,
  case_id uuid not null,
  case_sequence bigint not null,
  event_id uuid not null,
  event_type text not null,
  event_schema text not null,
  occurred_at timestamptz not null,
  recorded_at timestamptz not null default clock_timestamp(),
  actor jsonb not null,
  command_id uuid not null,
  authority jsonb not null,
  payload jsonb not null,
  evidence_refs jsonb not null,
  previous_event_digest text,
  event_digest text not null,
  primary key (tenant_id, case_id, case_sequence),
  unique (event_id),
  unique (tenant_id, case_id, event_digest)
);

```

An audit exporter creates ordered NDJSON or Parquet ledger segments, a manifest of event ranges and digests, and a service signature. It writes them to an immutable Blob container. Verification replays canonicalization, per-case chains, segment Merkle or manifest digests, signer trust, and sequence continuity.

## 14. Event schemas and evolution

Events are immutable facts expressed in the schema version effective when accepted. Consumers register supported versions. Additive compatible changes use a new minor schema. A semantic or structural breaking change uses a new event version and an explicit upcaster for projections.

The original bytes and digest never change. Upcasting is a read transformation with its own version, test fixtures, and diagnostics. Audit views can display original and normalized forms. A projection rebuild records the upcaster set used so results are reproducible.

Events describe domain meaning, not table deltas. `finding.created`, `claim.accepted_as_fact`, `contradiction.resolved`, and `case.dispositioned` remain meaningful after the physical database model changes.

## 15. Evidence object model

Evidence is first-class, immutable source material with provenance and policy. A database row stores metadata and references a content-addressed artifact.

```
{
  "evidenceId": "ev_01...",
  "schemaVersion": "evidence.v3",
  "tenantId": "tenant_01...",
  "caseId": "case_01...",
  "type": "screening_result",
  "source": {
    "type": "T00L",
    "providerBindingDigest": "sha256:...",
    "dataset": "sanctions-global",
    "datasetVersion": "2026-09-17",
    "capability": "screening.sanctions.search@3.2.0",
    "retrievedAt": "...",
    "sourceEffectiveAt": "..."
  },
  "subjectRefs": ["subject_01..."],
  "trust": {"tier": "LICENSED_PROVIDER", "instructionTrust": "UNTRUSTED_DATA"},
  "integrity": {
    "rawDigest": "sha256:...",
    "normalizedDigest": "sha256:...",
    "artifactRef": "artifact://evidence/sha256/..."
  },
  "classification": "CONFIDENTIAL",
  "retentionClass": "KYC_REGULATED",
  "residency": "US",
  "createdBy": {"executionId": "exec_01..."}
}
```

Evidence is not destructively edited. A correction, refreshed source, better scan, or changed normalization creates a new Evidence object linked by SUPERSEDES, CORRECTS, DERIVED\_FROM, or REFRESHES. The old object remains available under retention policy.

## 16. Evidence intake lifecycle

Evidence intake separates bytes from business acceptance.

1. Request a staged upload or open a trusted service stream.
2. Authenticate source and validate declared tenant, case, type, classification, region, media type, and size.
3. Scan active content and malware where applicable; quarantine failures.
4. Compute content digest while streaming and compare any declared digest.
5. Write bytes to a temporary non-authoritative path with encryption.
6. Extract safe technical metadata; do not treat extracted text as instruction.
7. Apply retention assignment and, where required, immutable-storage policy.
8. Promote or copy to the content-addressed artifact path and verify server-side properties.
9. Register Artifact and Evidence metadata in PostgreSQL.
10. Emit evidence.registered through the outbox.
11. A BusinessCommand links the verified Evidence to the case and produces a material ledger event.

An uploaded blob alone is not accepted evidence. A registered Evidence object alone does not alter case truth. A case command must verify admissibility and create the relationship.

## 17. Blob artifact layout

Use separate storage accounts or containers for regulated evidence, execution artifacts, temporary intake, quarantine, and audit exports so retention and access policies do not conflict.

```
staging/{tenant}/{upload-id}
quarantine/{tenant}/{artifact-id}
evidence/sha256/{first2}/{next2}/{full-digest}
normalized/sha256/{first2}/{next2}/{full-digest}
executions/{year}/{month}/{execution-id}/{artifact-digest}
ledger-export/{year}/{month}/{day}/{segment-id}
manifests/{artifact-or-segment-id}.json
```

The database stores an opaque artifact:// reference, storage account identity, container, blob name, version ID where used, content digest, length, media type, encryption metadata, and immutability state. Client-facing APIs never return an unrestricted storage URI. Downloads use the service as authorization broker or short-lived user-delegation SAS scoped to a single object and purpose.

## 18. Immutable storage, retention, and legal hold

Azure immutable Blob Storage supports time-based retention and legal holds that protect objects from modification or deletion: [Immutable storage for Blob data](#). Use version-level WORM for mixed retention when feature compatibility fits, or container-level WORM when a whole container shares one policy.

| Retention class   | Planning baseline                                 | Immutability treatment                                  |
|-------------------|---------------------------------------------------|---------------------------------------------------------|
| KYC_REGULATED     | Case closure plus seven years, subject to counsel | Locked time-based retention after validation            |
| DECISION_RECORD   | Seven years minimum                               | Locked WORM and signed manifest                         |
| EXECUTION_SUPPORT | Two years unless attached to decision             | Lifecycle policy; WORM for promoted evidence            |
| TRANSIENT_UPLOAD  | 24 hours                                          | No WORM; automatic deletion after validation or failure |
| QUARANTINE        | 30 to 90 days                                     | Restricted access; policy-driven disposal               |
| LEGAL_HOLD        | Until authorized release                          | Legal hold tag plus underlying retention                |

Legal hold is a controlled business process. A hold command identifies matter, scope, authority, effective time, and custodians; applies the storage control; records results and failures; and appends ledger/audit records without revealing restricted legal details to general case readers.

Locked retention can be extended but not shortened. Storage feature interactions, including versioning, soft delete, point-in-time restore, hierarchical namespace, and failover behavior, must be validated before production. Blob inventory and daily reconciliation verify policy assignment.

## 19. Evidence, Claim, Fact, and Finding

These objects have different epistemic meaning.

| Object   | Meaning                                                        | Example                                           |
|----------|----------------------------------------------------------------|---------------------------------------------------|
| Evidence | What a source returned or what was captured                    | Provider response lists John A Smith              |
| Claim    | A proposition asserted by a source, model, transform, or human | Candidate date of birth is 1978-03-22             |
| Fact     | A claim accepted under a named policy for a purpose and time   | Subject DOB accepted as 1978-03-22 for onboarding |

| Object        | Meaning                                                      | Example                                               |
|---------------|--------------------------------------------------------------|-------------------------------------------------------|
| Finding       | Workflow-relevant interpretation supported by facts/evidence | Possible sanctions match, confidence 0.71             |
| Assumption    | Explicit proposition used with incomplete support            | Address normalization likely refers to same residence |
| Contradiction | Material conflict between claims or evidence                 | Two credible DOBs disagree                            |

An LLM cannot silently turn evidence into fact. Claim extraction creates lineage. Fact acceptance cites claims, acceptance policy, purpose, validity interval, and actor. A finding cites facts/evidence, method, confidence semantics, uncertainty, and policy relevance.

## 20. Bitemporal facts

Facts support valid time and system time so the service can answer what was true in the domain and what the organization believed at a given decision time.

```
{
  "factId": "fact_01...",
  "subjectRef": "subject_01...",
  "predicate": "date_of_birth",
  "value": "1978-03-22",
  "validTime": {"from": null, "to": null},
  "systemTime": {"recordedAt": "2026-09-17T18:41:12Z", "supersededAt": null},
  "basis": ["claim_17", "claim_18"],
  "acceptancePolicyRef": "identity-fact-policy@3.2.1",
  "purpose": "KYC_ONBOARDING",
  "status": "ACTIVE"
}
```

A correction closes the prior fact's system interval and creates a successor plus events. It does not falsify what was known when an earlier decision was made. Decision packets pin fact revisions and policy snapshots.

## 21. Lineage graph

Typed edges create the epistemic chain:

```
Evidence -> ASSERTS -> Claim
Evidence -> SUPPORTS / CONTRADICTS -> Claim
Claim -> ACCEPTED_AS -> Fact
Fact -> SUPPORTS -> Finding
Assumption -> USED_BY -> Finding
Contradiction -> CONCERNS -> Claim / Fact / Evidence
Finding -> SUPPORTS / OPPOSES -> DecisionRecommendation
DecisionRecommendation -> EVALUATED_BY -> DecisionGateResult
DecisionGateResult -> AUTHORIZES / DENIES -> FinalDisposition
ExecutionRecord -> PRODUCED -> Claim / Finding / Recommendation
HumanReview -> CONFIRMS / OVERRIDES -> governed object
```

Edges carry source and target revision, predicate version, actor, time, method, confidence where relevant, policy reference, and creation event. The graph is stored relationally and queried through purpose-specific APIs. A graph or search projection may accelerate traversals, but PostgreSQL edges and Blob evidence remain canonical.

## 22. Assumptions and contradictions

Assumptions make incomplete support visible. States are UNVERIFIED, CORROBORATED, REJECTED, WAIVED, and SUPERSEDED. High-materiality unverified assumptions can block selected dispositions.

Contradictions are first-class. They reference the conflicting objects, values or interpretations, materiality, detection method, status, and required resolution class. States are OPEN, UNDER\_INVESTIGATION, RESOLVED, ACCEPTED\_RISK, and SUPERSEDED.

Resolution creates a new event and links evidence, rationale, actor, policy, and resulting accepted or rejected claims/facts. The contradiction remains visible. Policy can require no material open contradiction before APPROVED or REJECTED dispositions.

## 23. Decision chain

The Decision Agent or analyst proposes an outcome; it does not make the legally effective decision by itself.

```

DecisionPacket
|
DecisionRecommendation
|
Deterministic DecisionGateResult
|
Human approval where required
|
FinalDisposition

```

The DecisionPacket pins the case sequence, policy snapshot, evidence and fact revisions, findings, contradictions, assumptions, completed WorkItems, QA result, and execution references. The gate evaluates completeness, admissibility, required checks, policy rules, contradiction state, authority, and review obligations.

FinalDisposition records effective outcome, time, recommendation, gate result, policy snapshot, authorized actor, review/override record, and follow-up obligations. Changing an effective disposition creates reversal or supersession events under explicit policy.

## 24. Human review

Human review uses the same WorkItem, evidence, command, and ledger model as automated actors. An analyst interface reads a purpose-limited case view and submits commands such as ConfirmClaim, RejectEvidence, ResolveContradiction, OverrideRecommendation, or ApproveDisposition.

The service records authenticated principal, active role, authentication context, queue assignment, viewed evidence set, action time, rationale, policy/override code, and resulting events. Sensitive free-text notes use a governed schema and classification. A user interface cannot write final tables directly.

Dual control uses separate assignments and principals. The second approver sees the same pinned DecisionPacket or is warned that the case changed and must re-review.

## 25. Query and read models

The query API returns explicit views rather than exposing tables:

- CaseSummaryView for queues and dashboards.
- CaseOperationalView for current WorkItems and effective state.
- CaseDecisionView for recommendations, gates, evidence sufficiency, and dispositions.
- CaseAuditView for ordered events and authority.
- EvidenceView for metadata, lineage, and authorized artifact access.
- AsOfCaseView reconstructed at a ledger sequence or system time.

Every response includes case sequence, view schema, policy/authorization context, generated time, and ETag. Field-level filtering occurs before serialization. Cached views bind tenant, purpose, roles, policy, sequence, and classification.

Pagination uses stable keys such as (case\_sequence, event\_id) rather than offset. Timeline queries can select domain-effective time or recorded order and clearly label the choice.

## 26. Search and retrieval

Azure AI Search may index approved evidence text, subjects, findings, and ledger summaries for authorized retrieval. It is a derived view. PostgreSQL remains the metadata and relationship authority; Blob remains the artifact authority.

Index documents include tenant, case, object and revision IDs, source digest, classification, allowed security principals or groups, retention state, trust label, language, and projection generation. Queries apply security filters and purpose before ranking.

Deleting an index does not lose evidence. Rebuild reads the outbox/event history and current authoritative objects. Projection reconciliation compares source sequence and digest. Vector embeddings are derived artifacts with model/version provenance and cannot be treated as source evidence.

## 27. APIs

| Endpoint                        | Semantics                                      | Concurrency and authority                          |
|---------------------------------|------------------------------------------------|----------------------------------------------------|
| POST /v1/cases                  | Create case from validated intake              | Idempotency key; tenant and case-type permission   |
| GET /v1/cases/{id}              | Read purpose-specific current view             | Sequence/ETag; field authorization                 |
| POST /v1/cases/{id}/commands    | Submit BusinessCommand                         | Idempotency, preconditions, plan/policy authority  |
| GET /v1/cases/{id}/events       | Read ordered ledger                            | Stable cursor; audit permission                    |
| GET /v1/cases/{id}:asOf         | Reconstruct governed historical view           | Sequence/time and schema/upcaster set              |
| POST /v1/evidence:stage         | Create staged upload authorization             | Type, size, media, region, classification          |
| POST /v1/evidence:register      | Verify and register immutable Evidence         | Content digest, provenance, retention, scan status |
| GET /v1/evidence/{id}           | Read metadata and lineage                      | Case, purpose, classification, legal restrictions  |
| POST /v1/evidence/{id}:download | Broker single-artifact access                  | Short-lived scoped authorization and audit         |
| POST /v1/legal-holds            | Apply governed hold                            | Privileged role, matter scope, dual approval       |
| POST /v1/ledger:verify          | Verify sequence, hashes, manifests, signatures | Auditor/service permission                         |

Mutation APIs require correlation ID and idempotency key. Command payloads are signed or authenticated through Entra; transport signatures do not replace actor authorization. Responses use stable error codes and do not disclose cross-tenant existence.

## 28. Events and outbox

The transaction writes an outbox record for every publishable CaseEvent. A publisher sends integration events only after commit. Service Bus handles required internal processing and backpressure; Event Grid distributes loosely coupled CloudEvents to analytics, notification, or downstream domains.

Event payloads carry event ID, tenant/case opaque references, sequence, type, schema, recorded time, subject references, classification, source event digest, and a retrieval link or minimal approved projection. Restricted evidence content is never placed directly on a general event bus.

Consumers are idempotent by event ID and track source sequence. Gaps pause a strict projection and trigger replay. Dead-letter repair preserves the original event and records operator action. Publishing failure cannot roll back accepted case state; the outbox retries until delivered.

## 29. Security and authorization

The API uses Entra tokens and managed identities. Human access uses Conditional Access and privileged roles where required. Service identities are separated for command processing, evidence intake, projection, audit export, legal hold, and support diagnostics.

Authorization combines tenant, case assignment, actor role, command or view, purpose, data class, jurisdiction, legal restriction, PermissionEnvelope for agents, and policy snapshot. PostgreSQL row-level security may provide defense in depth, but application authorization remains mandatory and connection pooling must preserve identity context correctly.

Database administration, storage administration, key administration, legal hold, application deployment, policy ownership, and audit verification are separated. Production support uses time-bound PIM roles and audited break-glass procedures. No role can both alter ledger data and erase the independent audit export.

## 30. Encryption and key management

All service traffic uses TLS. PostgreSQL, Blob, backup, and logs are encrypted at rest. Customer-managed keys are used when required by the security or contractual profile; otherwise Microsoft-managed encryption remains acceptable only by approved decision.

Use separate Key Vault keys for evidence storage, audit export, and other regulated stores when cryptographic separation is required. Key Vault uses private endpoints, RBAC, purge protection, rotation, and monitored access. Storage identities receive only required wrap/unwrap or data-plane roles.

Artifact-level envelope encryption can be added for tenant-dedicated keys or crypto-shredding use cases, but legal retention and recoverability must be reconciled before selecting it. Key deletion is never used as an informal erasure mechanism.

## 31. Privacy, minimization, and erasure

The current case stores only data needed for the declared KYC purpose. Sensitive identifiers use tokenization or dedicated protected columns; broad JSONB payloads are not a substitute for classification. APIs return masked values unless the purpose requires full disclosure.

Retention and privacy requests are policy decisions, not direct deletes. The service evaluates legal basis, active case, regulatory retention, legal hold, downstream systems, and evidence obligations. When erasure is permitted, it creates a governed command, ledger event, tombstone or redaction artifact, projection deletion, and verification report.

An immutable ledger can preserve minimal proof that a governed action occurred while removing or cryptographically isolating content that policy permits to erase. The exact treatment requires privacy and legal approval; the architecture does not promise deletion that WORM policy prevents.

## 32. Azure PostgreSQL configuration

Use Azure Database for PostgreSQL Flexible Server with private access, Microsoft Entra authentication, TLS enforcement, zone-redundant high availability where supported, automated backups, and tested point-in-time restore. Microsoft documents zone-redundant HA and business continuity options: [PostgreSQL business continuity](#). Microsoft Entra principals and managed identities can authenticate without long-lived database passwords: [PostgreSQL Entra authentication](#).

| Setting        | Production baseline                                                                 |
|----------------|-------------------------------------------------------------------------------------|
| Region         | East US 2 primary; approved US recovery region                                      |
| Compute        | General Purpose 8 vCores initial; validate memory and IOPS under load               |
| Storage        | 1 TiB initial with growth alerts and measured IOPS/autogrow policy                  |
| HA             | Zone redundant where available for selected tier/region                             |
| Backup         | 35-day operational retention planning baseline; geo-redundant backup if approved    |
| Authentication | Entra-only where operationally feasible; managed identities for services            |
| Connectivity   | Private access/private endpoint pattern, private DNS, public network disabled       |
| Pooling        | PgBouncer or application pool with transaction mode validated against session needs |
| Extensions     | Allowlisted only; no extension becomes an unreviewed execution path                 |
| Diagnostics    | Connections, locks, transactions, storage, WAL, replication, query performance      |

Partition case\_events, command receipts, and high-volume metadata by time and/or stable tenant strategy after load testing. Avoid per-tenant tables. Index by tenant and case first, then sequence, status, subject, work queue, or temporal predicate. JSONB GIN indexes require measured query value.

## 33. Azure Blob configuration

Use General Purpose v2 accounts dedicated by environment and data purpose. Public network access and shared-key authorization are disabled. Services use managed identity and private endpoints for Blob and Data Lake endpoints only when needed. Enable secure transfer, minimum TLS, versioning/soft delete where compatible, change feed or inventory where required, diagnostic settings, and Defender controls according to policy.

Evidence storage uses locally redundant, zone-redundant, or geo-redundant design selected from RPO, WORM compatibility, region policy, and recovery testing. Regulatory artifacts use an immutability design tested before locking. Audit exports use a separate account and administration boundary.

Lifecycle rules move eligible content from hot to cool/cold/archive tiers according to access and legal requirements. A restore request from archive is an asynchronous governed operation; the Case Service exposes availability state rather than failing mysteriously.

## 34. Service topology and sizing

| Azure resource             | Production baseline                                                        | Initial configuration                                                                       |
|----------------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| APIM                       | Internal/private production ingress                                        | Entra validation, schema/size limits, quotas, diagnostics without payloads                  |
| Container Apps             | Case API, command processor, evidence service, ledger exporter, projectors | Zone redundancy where supported; API min 3; worker scale by queue                           |
| PostgreSQL Flexible Server | Authoritative current state and ledger metadata                            | Zone HA, Entra auth, private access, PITR, query store/monitoring                           |
| Evidence Storage           | Content-addressed artifacts                                                | Private endpoint, immutable policies, versioning/soft delete compatibility, CMK if required |
| Audit Storage              | Independent immutable ledger exports                                       | Separate identity/admin boundary, locked WORM after validation                              |
| Service Bus Premium        | Internal required processing                                               | Commands/events where asynchronous, DLQ, duplicate detection, private endpoint              |
| Event Grid                 | Integration facts                                                          | CloudEvents, subscriptions by approved classification, dead-letter destination              |
| Key Vault Premium          | Keys and exceptional secrets                                               | Private endpoint, purge protection, rotation, separate roles                                |
| Azure AI Search            | Derived authorized retrieval                                               | Private endpoint, managed identity, security filters, rebuildable indexes                   |
| Azure Monitor              | Traces, metrics, redacted logs                                             | Application Insights and Log Analytics with access and retention controls                   |

Initial service sizing is Case API min/max replicas 3/40, command workers 3/60, evidence intake 2/30, ledger exporter 2/10, and projectors 2/50. Autoscale on HTTP concurrency, Service Bus depth, outbox lag, and evidence throughput. Load tests determine final database, storage, and network capacity.

## 35. Observability and reconciliation

OpenTelemetry spans cover command receipt, authorization, aggregate load, evidence validation, policy evaluation, mutation, ledger append, commit, outbox publish, artifact registration, download authorization, and projection. Logs contain IDs, digests, classifications, sequences, rule codes, duration, and outcome; they do not contain evidence bytes, SSNs, documents, or unrestricted free text.

| Objective                 | Proposed target                                        | Alert                                  |
|---------------------------|--------------------------------------------------------|----------------------------------------|
| Case command availability | 99.95 percent monthly                                  | Multi-window burn rate                 |
| Case query availability   | 99.95 percent monthly                                  | Multi-window burn rate                 |
| Simple command latency    | p95 under 400 ms excluding external preparation        | p95 above 750 ms                       |
| Query latency             | p95 under 300 ms for standard views                    | p95 above 500 ms                       |
| Ledger atomicity          | 100 percent accepted mutations have event and sequence | Any reconciliation mismatch            |
| Evidence integrity        | 100 percent artifact digests verify                    | Any digest or length mismatch          |
| Outbox propagation        | 99 percent under 30 seconds                            | Oldest unpublished row over 60 seconds |
| Projection completeness   | 99.9 percent under two minutes                         | Sequence gap or lag over threshold     |

| Objective              | Proposed target                              | Alert                   |
|------------------------|----------------------------------------------|-------------------------|
| Legal hold application | 100 percent verified across scoped artifacts | Any partial application |

Daily reconciliation checks current case sequence against ledger head, event continuity and hash chain, command receipts, outbox delivery, evidence metadata against Blob properties and digest samples, immutability state, legal holds, search projection generation, and audit-export manifests. Discrepancies create incidents and block destructive lifecycle actions.

## 36. Backup, recovery, and regional continuity

High availability handles local faults; disaster recovery handles regional loss. Proposed targets are RPO at most 5 minutes and RTO at most 60 minutes for current case and ledger metadata, with evidence-artifact objectives defined by storage redundancy and immutability design. No failover plan assumes that immutable-policy changes automatically replicate without verification.

Recovery sequence:

1. Declare incident and freeze command intake or route to the approved recovery stamp.
2. Restore/promote PostgreSQL and verify case/ledger sequence consistency.
3. Establish evidence and audit-storage availability; verify manifests and legal holds.
4. Reconcile outbox and replay unpublished events idempotently.
5. Rebuild caches, search indexes, and graph projections.
6. Verify signing keys, DNS, identities, private endpoints, and service configuration.
7. Run read-only integrity checks and controlled command canary.
8. Resume traffic and record recovery events.

Quarterly exercises restore the database to an isolated subscription, verify random artifact digests, reconstruct a case as of historical sequence, validate ledger chains and audit signatures, apply and release a test legal hold, and rebuild search from authority.

## 37. KYC domain rules

KYC commands and events include subject creation/correction, consent capture, identity evidence registration, claim extraction, fact acceptance, screening finding, contradiction detection/resolution, enhanced due diligence initiation, risk assessment, recommendation, QA result, gate result, human review, disposition, periodic review, and closure.

Required invariants include:

- APPROVED requires mandatory identity and screening evidence at required freshness and assurance.
- REJECTED cites a permissible policy basis and evidence; model sentiment alone is prohibited.
- High-confidence sanctions or PEP matches cannot be silently downgraded by a model.
- Material open contradictions block configured dispositions.
- Vendor unavailability and no match are distinct states.
- Evidence freshness and dataset version are preserved at decision time.
- A later screening update does not rewrite the earlier decision packet.
- Human overrides require authority, reason code, rationale, and, where configured, dual control.
- SSNs, identity documents, biometrics, and sensitive vendor payloads use explicit classification, purpose, access, retention, and telemetry rules.

## 38. CI/CD and database change management

Application images are built once, scanned, signed, and deployed by digest. Database migrations are versioned, backward compatible, and executed by a dedicated migration identity. Runtime services never have DDL rights.

```
pull request
-> unit + schema + policy + migration tests
-> event/command compatibility tests
-> ledger canonicalization and hash-chain fixtures
-> build + SBOM + vulnerability + OCI signature
-> deploy isolated nonprod
-> transactional, concurrency, idempotency, and evidence tests
-> backup/restore and projection rebuild tests
-> performance and failover tests
-> canary by tenant/case type
-> staged production rollout
```

Expand-and-contract sequence adds new schema and dual-readable code, backfills projections or current rows with checkpoints, switches readers/writers after verification, and removes obsolete structures only after retained versions no longer require them. Ledger source events are never rewritten during migration.

## 39. Implementation plan

| Phase                           | Duration | Deliverables and exit criteria                                                                             |
|---------------------------------|----------|------------------------------------------------------------------------------------------------------------|
| 0. Semantics and threat model   | 2 weeks  | Aggregate boundary, canonical records, object semantics, retention assumptions, threat model, ADRs         |
| 1. Azure data foundation        | 3 weeks  | PostgreSQL, Storage, private networking, identities, Key Vault, Service Bus/Event Grid, monitoring, backup |
| 2. Case core                    | 5 weeks  | Cases, subjects, commands, domain service, query views, idempotency, concurrency, KYC lifecycle            |
| 3. Ledger                       | 4 weeks  | Event envelope, sequence, hash chain, schema registry, outbox, audit export and verification               |
| 4. Evidence Store               | 5 weeks  | Staging, scanning, digesting, Blob layout, registration, metadata, download brokerage, retention           |
| 5. Epistemic graph              | 4 weeks  | Claim, fact, finding, assumption, contradiction, lineage APIs, bitemporal semantics                        |
| 6. Decision and review          | 4 weeks  | DecisionPacket, recommendation, deterministic gate, human review, override, disposition                    |
| 7. Projections and integrations | 3 weeks  | Search, timelines, analytics/events, projection rebuild and reconciliation                                 |
| 8. KYC pilot                    | 4 weeks  | Identity/screening flows, EDD, QA, human review, disposition, evidence and audit package                   |
| 9. Hardening and DR             | 3 weeks  | Load, chaos, restore, regional recovery, legal hold exercise, SLOs and runbooks                            |

## 40. Prioritized engineering backlog

| ID     | Work item                                  | Acceptance criterion                                                                      |
|--------|--------------------------------------------|-------------------------------------------------------------------------------------------|
| CS-01  | Define Case and KYC lifecycle schemas      | Domain and policy owners approve commands, states, invariants, and versioning             |
| CS-02  | Implement command gateway                  | Auth, schema, evidence, policy, concurrency, idempotency, and domain checks are atomic    |
| CS-03  | Implement aggregate persistence            | Relational current state advances only through accepted commands                          |
| CS-04  | Add entity revisions and merge semantics   | Append, rebase, revalidate, recompute, and exact-match behaviors pass conflict tests      |
| CL-01  | Implement ordered CaseEvent append         | Every material mutation shares the transaction and consecutive sequence                   |
| CL-02  | Implement canonical event digest and chain | Independent verifier detects alteration, deletion, insertion, and reordering              |
| CL-03  | Build schema registry and upcasters        | All retained event versions rebuild projections deterministically                         |
| CL-04  | Export signed immutable ledger segments    | Audit account verifies sequence, hashes, manifest, signature, and WORM state              |
| EV-01  | Build staged evidence intake               | Type, size, malware, digest, region, classification, and source validation fail closed    |
| EV-02  | Implement content-addressed artifact store | Duplicate bytes deduplicate safely within allowed scope and always verify digest          |
| EV-03  | Implement retention and legal hold         | Scoped application, verification, audit, conflict handling, and release workflow pass     |
| EV-04  | Implement evidence download broker         | Single-object, purpose-limited, short-lived access is logged and revocable                |
| EP-01  | Model Claim/Fact/Finding semantics         | An evidence-to-fact transition requires explicit acceptance policy and lineage            |
| EP-02  | Model assumptions and contradictions       | Material unresolved state blocks configured dispositions and remains historically visible |
| DC-01  | Implement decision chain                   | Recommendation cannot become disposition without matching gate and authority              |
| OPS-01 | Build reconciliation service               | Detects sequence, outbox, artifact, WORM, hold, projection, and export mismatch           |
| OPS-02 | Prove restore and rebuild                  | Historical case, ledger, evidence, and derived projections recover within targets         |
| KYC-01 | Deliver KYC pilot                          | End-to-end onboarding and review produce complete decision provenance and audit package   |

## 41. Verification strategy

Unit tests cover command schemas, authorization decisions, domain invariants, merge semantics, idempotency, event canonicalization, sequence allocation, hash chains, event upcasting, artifact digests, lineage rules, bitemporal facts, contradictions, gate rules, and retention assignment.

Property tests generate command interleavings and assert serializable domain outcomes, no sequence gaps, no duplicate semantic effects, and stable event digests. Fuzz tests target JSONB schemas, file metadata, archive formats, OCR text, image parsers, external identifiers, event payloads, and query filters.

Integration tests use actual Azure PostgreSQL, Blob, Key Vault, Service Bus, Event Grid, Container Apps, and AI Search in an isolated subscription. They verify private networking, managed identity, transaction rollback, outbox replay, WORM and legal-hold behavior, archive restore, projection rebuild, and audit export.

End-to-end KYC tests register identity and screening evidence, create claims and facts, detect and resolve a contradiction, produce a finding and recommendation, run the decision gate, perform human review, commit a disposition, reconstruct the case as of the decision sequence, and verify every artifact and ledger digest.

Fault tests kill services during commit, duplicate commands, reorder messages, fail Blob after staging, fail PostgreSQL before/after commit, block outbox publishing, corrupt a test artifact, create a projection gap, expire download authorization, and partially apply a test hold. No accepted state may lack its ledger event, and no ledger event may describe an uncommitted mutation.

## 42. Operational runbooks

Required runbooks cover command latency, database lock contention, duplicate-key conflict, sequence mismatch, ledger-chain verification failure, outbox backlog, projection gap, evidence scan timeout, digest mismatch, quarantined artifact, Blob unavailability, archive restore, WORM policy mismatch, legal-hold partial application, Key Vault failure, database failover, PITR, regional recovery, unauthorized access alert, privacy request, corrupt projection, and audit-export signature failure.

Each runbook states detection, case/customer impact, immediate containment, diagnostic queries, safe retry or replay, escalation, evidence preservation, approval requirements, and closure verification. Operators never edit current case, ledger, evidence metadata, or storage immutability state outside governed commands and administrative workflows.

## 43. Architecture decisions and definition of done

The initial decisions approve PostgreSQL as authority for current case and ledger metadata; one transaction for state, event, sequence, command receipt, and outbox; Blob as content-addressed evidence store; independent immutable ledger export; typed Evidence/Claim/Fact/Finding semantics; bitemporal facts; first-class assumptions and contradictions; BusinessCommands as the only write interface; semantic idempotency; and derived search/analytics projections.

Production pilot readiness requires all of the following:

- No application, agent, analyst UI, integration, or workflow service can update canonical case tables directly.
- Every accepted material mutation has a consecutive ledger event, actor, command, authority, evidence references, prior digest, and event digest in the same transaction.
- Independent verification detects ledger alteration, deletion, insertion, duplication, and reordering.
- Evidence bytes are digest-addressed, encrypted, private, classified, retention-assigned, and linked through verified metadata.
- Locked retention and legal-hold flows are tested in the exact production storage configuration.
- Claim, fact, finding, assumption, contradiction, recommendation, gate, and disposition transitions have explicit lineage and policy.
- Concurrent commands produce approved merge, revalidation, recomputation, or conflict behavior without lost updates.

- Case reconstruction at historical sequence reproduces the effective state and decision packet.
- Search and analytics can be deleted and rebuilt from authoritative records.
- Backup, restore, regional recovery, artifact verification, and audit-export reconciliation meet approved RPO/RTO.
- KYC, compliance, privacy, legal, security, SRE, platform, and audit owners approve the evidence package and operating model.

The result is a durable case platform in which current truth, immutable history, source evidence, computational provenance, and operational telemetry remain connected but never conflated.